

Clean Architecture & Refactoring

Chapter 02 | Sub-Chapter 10

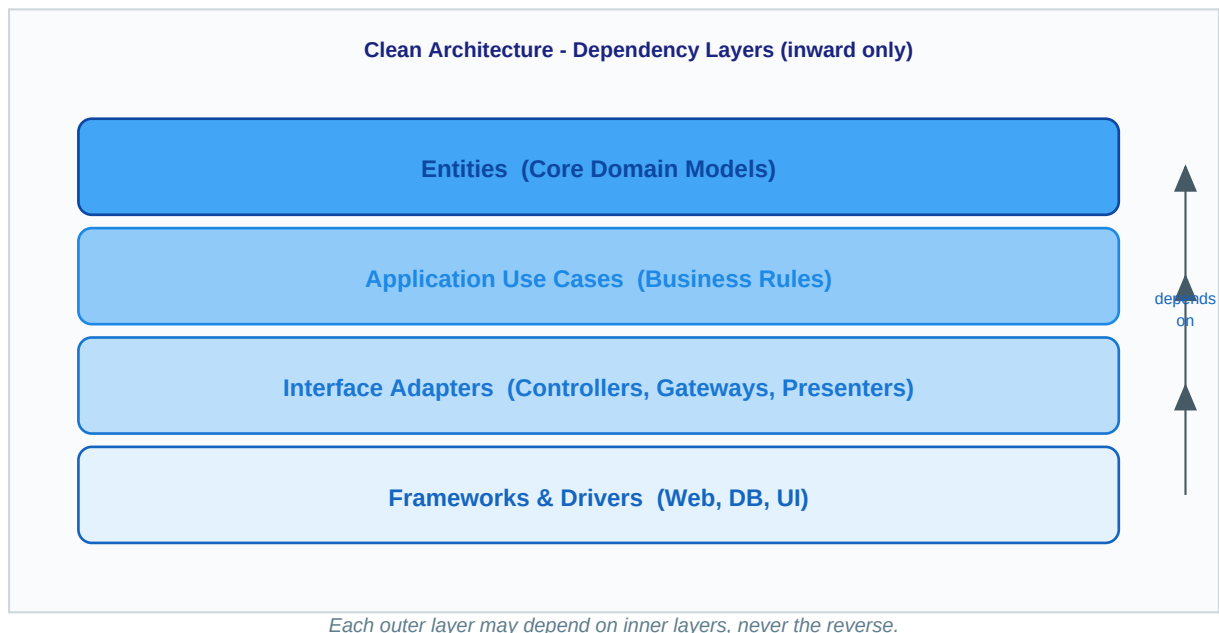
Beginner-Friendly Java Guide

Clean Architecture	Layers: Entity, UseCase, Interface, Framework
Hexagonal Architecture	Ports and Adapters pattern for testable design
CQRS	Separate Command (write) and Query (read) models
Code Smells	Identify God Class, Long Method, Feature Envy
Refactoring Techniques	Extract Method, Replace Conditional, Param Object
Testing Strategy	Unit / Integration / E2E pyramid approach

1 Clean Architecture

Dependency Rule: always point inward

Clean Architecture arranges code in concentric circles. Inner layers hold business rules; outer layers hold delivery mechanisms. The golden rule: source-code dependencies must always point inward, never outward.



Simple Java Example

```
// Entity (innermost layer)
public class Order {
    private String id;
    private double total;
    public boolean isValid() { return total > 0; }
}

// Use Case (depends only on Entity + interface)
public class PlaceOrderUseCase {
    private final OrderRepository repo; // interface, not a class!
    public void execute(Order o) {
        if (o.isValid()) repo.save(o);
    }
}

// Interface Adapter (outermost we show)
public class OrderController {
    private final PlaceOrderUseCase useCase;
    public void handle(HttpRequest req) {
        Order o = map(req);
        useCase.execute(o);
    }
}
```

Key Rules

- Entities know nothing about use cases, adapters, or frameworks.
- Use Cases depend only on Entities and abstract interfaces (ports).

- Interface Adapters convert data between use cases and the outside world.
- Frameworks and Drivers (Spring, Hibernate) live in the outermost ring.
- Always code to interfaces so inner layers stay independent of outer details.

Q1. What is the core rule of Clean Architecture?

Dependencies must point inward: outer layers depend on inner, never the reverse.

Q2. Where do database queries live in Clean Architecture?

In the Interface Adapters layer (Gateway/Repository implementations), not in Use Cases.

Q3. Why do Use Cases use interfaces instead of concrete classes?

So business logic stays independent of frameworks, databases, or delivery mechanisms.

Q4. What belongs in the Entity layer?

Enterprise-wide business objects and critical business rules that change rarely.

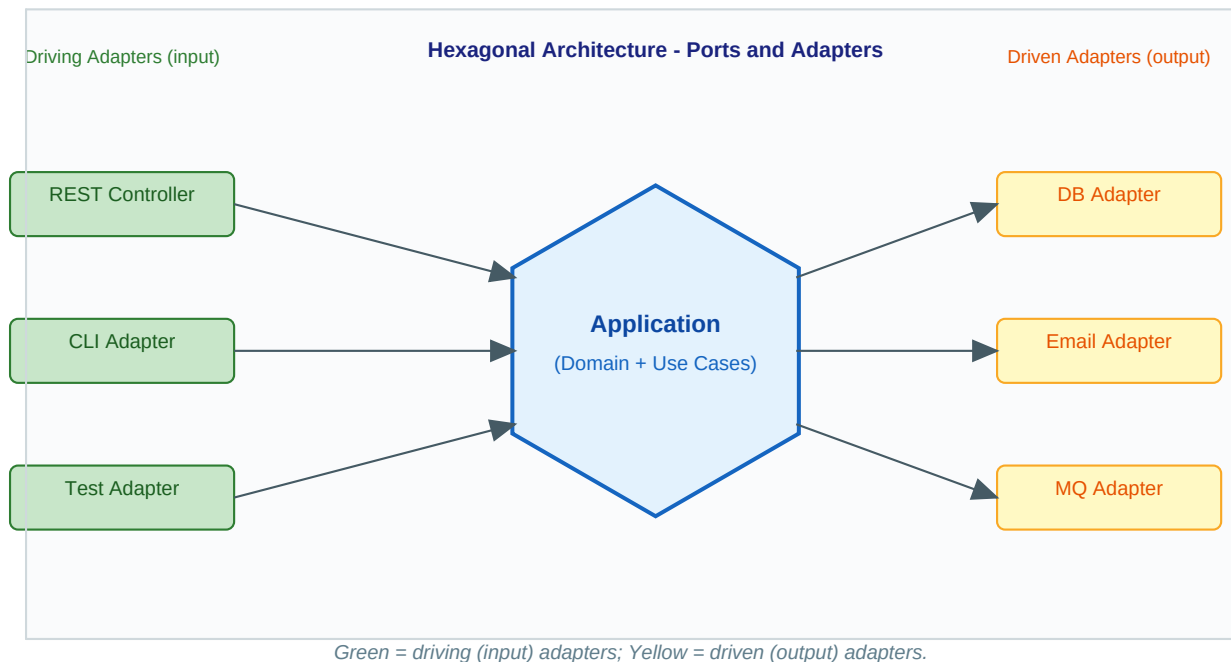
Q5. How does Clean Architecture improve testability?

Use Cases and Entities have no external dependencies, so they can be unit-tested in isolation.

2 Hexagonal Architecture

Ports and Adapters (Alistair Cockburn)

Hexagonal Architecture places the application at the center. Each side of the hexagon represents a port (interface). Adapters on the outside translate between external technology and the port contract. The application never knows which adapter is plugged in.



Simple Java Example

```
// Port (interface defined by the application)
public interface NotificationPort {
    void send(String userId, String message);
}

// Adapter (infrastructure detail, implements the port)
public class EmailAdapter implements NotificationPort {
    public void send(String userId, String msg) {
        // send real email via SMTP
    }
}

// Use Case depends only on the Port, not the adapter
public class RegisterUserUseCase {
    private final NotificationPort notifier;
    public void execute(User u) {
        userRepo.save(u);
        notifier.send(u.getId(), "Welcome!");
    }
}
```

Key Rules

- Ports are interfaces defined by the application, not by external tools.
- Adapters implement ports and contain all technology-specific code.
- Driving adapters call into the application (REST, CLI, tests).
- Driven adapters are called by the application (DB, email, queues).

- Swap any adapter without touching business logic - that is the key benefit.

Q1. What is a port in Hexagonal Architecture?

An interface that defines how the application communicates with the outside world.

Q2. What is the difference between a driving and a driven adapter?

Driving adapters initiate interaction (REST calls the app); driven adapters are called by the app (DB).

Q3. How does Hexagonal Architecture aid testing?

Replace real adapters with in-memory or mock adapters without changing any application code.

Q4. Is Hexagonal Architecture the same as Clean Architecture?

They share the same dependency-inversion principle but Hexagonal focuses on ports/adapters explicitly.

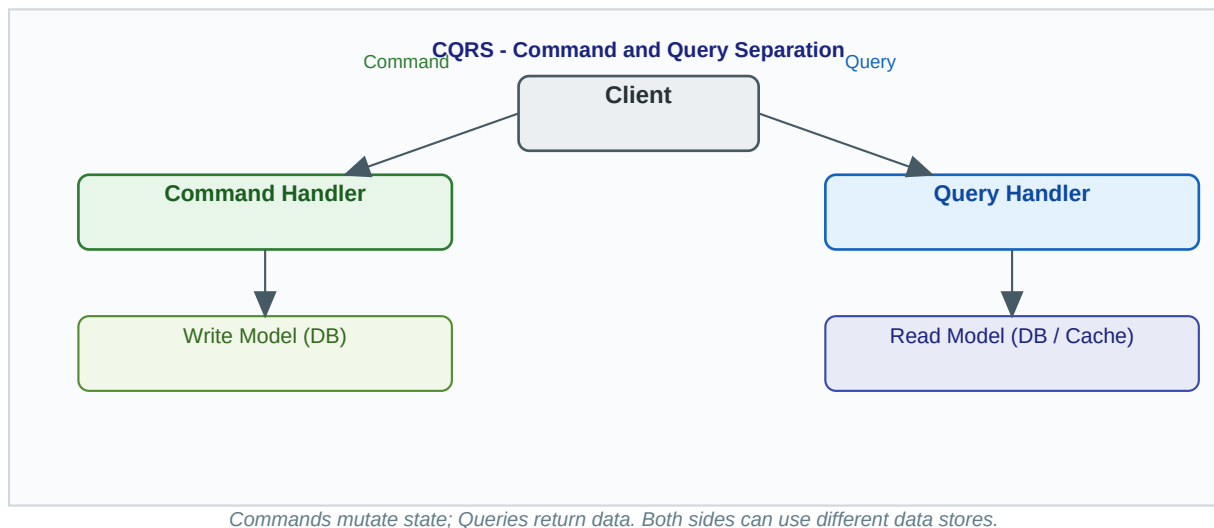
Q5. Where would you put an HTTP controller in this pattern?

In a driving adapter on the left side of the hexagon, implementing an input port interface.

3 CQRS

Command Query Responsibility Segregation

CQRS separates the model used for writes (Commands) from the model used for reads (Queries). Commands change state but return nothing. Queries return data but change nothing. This makes each side simple and independently scalable.



Simple Java Example

```
// Command (intent to change state - no return value)
public class CreateOrderCommand {
    public final String customerId;
    public final double amount;
}

// Command Handler (write side)
public class CreateOrderHandler {
    public void handle(CreateOrderCommand cmd) {
        Order o = new Order(cmd.customerId, cmd.amount);
        orderRepo.save(o);
    }
}

// Query (read side - no mutation allowed)
public class GetOrderQuery {
    public OrderDTO execute(String orderId) {
        return orderReadRepo.findDtoById(orderId);
    }
}
```

Key Rules

- Commands change state and return void (or an ID at most).
- Queries return data and must have zero side-effects.
- Read and write models can use completely different database schemas.
- Event sourcing pairs naturally with CQRS: commands produce events, queries read projections.
- Apply CQRS where read/write loads differ significantly - not everywhere by default.

Q1. What does CQRS stand for?

Command Query Responsibility Segregation.

Q2. Can the read and write databases be different in CQRS?

Yes. A common pattern uses a relational DB for writes and Elasticsearch or a cache for reads.

Q3. What is the main benefit of separating commands from queries?

Each side can be optimized, scaled, and changed independently.

Q4. Does every project need CQRS?

No. It adds complexity. Use it when read and write workloads have very different requirements.

Q5. What is a projection in CQRS?

A read-optimized view of data built by replaying events or syncing from the write model.

4 Code Smells

Signs that code needs refactoring

Code smells are surface indicators of deeper design problems. They do not cause bugs directly but make code harder to read, test, and extend. Recognising them is the first step to refactoring.

God Class - One class that does everything - hundreds of methods, all responsibilities mixed.

Long Method - A method so long it is hard to understand. Aim for under 20 lines per method.

Feature Envy - A method that accesses data of another class more than its own class.

Data Clumps - The same group of fields/parameters always appearing together - they want to be a class.

Before - God Class smell

```
// BAD: one class handles everything
public class AppManager {
    public void saveUser(User u) { ... }
    public void sendEmail(String to, String msg) { ... }
    public void generateInvoice(Order o) { ... }
    public void processPayment(Card c) { ... }
    public void logActivity(String event) { ... }
    // ...50 more unrelated methods
}
```

After - responsibilities split into focused classes

```
// GOOD: each class has exactly one responsibility
public class UserService { void save(User u) { ... } }
public class EmailService { void send(String to, String m) { ... } }
public class InvoiceService { void generate(Order o) { ... } }
public class PaymentService { void process(Card c) { ... } }
public class AuditLogger { void log(String event) { ... } }
```

Key Rules

- God Class: split by Single Responsibility - each class owns one concern.
- Long Method: extract cohesive code blocks into well-named private methods.
- Feature Envy: move the method to the class whose data it uses most.
- Data Clumps: group repeated parameter sets into a dedicated value object.
- Use SonarQube or PMD to detect smells automatically in Java projects.

Q1. What is a God Class?

A class that has taken on too many responsibilities, violating the Single Responsibility Principle.

Q2. How do you fix Feature Envy?

Move the method to the class it accesses most, or extract shared logic into a new service.

Q3. Why are Data Clumps a problem?

Scattered groups of related fields lead to duplication and make changes error-prone.

Q4. Is a long method always a code smell?

Not always, but methods longer than 20-30 lines are often a sign of mixed concerns.

Q5. What tools detect code smells in Java projects?

SonarQube, Checkstyle, and PMD are popular static analysis tools for Java.

5 Refactoring Techniques

Improve design without changing behaviour

Refactoring means changing the internal structure of code without altering its external behaviour. Always run tests before and after each refactoring step to stay safe.

Technique 1 - Extract Method

Pull a logical block of code into its own named method.

Before

```
public void processOrder(Order o) {
    if (o == null || o.getTotal() <= 0)
        throw new IllegalArgumentException("bad order");
    double tax = o.getTotal() * 0.2;
    o.setTax(tax);
    orderRepo.save(o);
}
```

After

```
public void processOrder(Order o) {
    validate(o);
    applyTax(o);
    orderRepo.save(o);
}
private void validate(Order o) {
    if (o == null || o.getTotal() <= 0)
        throw new IllegalArgumentException("bad order");
}
private void applyTax(Order o) { o.setTax(o.getTotal() * 0.2); }
```

Technique 2 - Replace Conditional with Polymorphism

Replace if/switch on type with subclass overrides.

Before

```
public double discount(String type, double price) {
    if (type.equals("VIP")) return price * 0.2;
    if (type.equals("Regular")) return price * 0.1;
    return 0;
}
```

After

```
public interface DiscountStrategy {
    double apply(double price);
}
public class VipDiscount implements DiscountStrategy {
    public double apply(double p) { return p * 0.2; }
}
public class RegularDiscount implements DiscountStrategy {
    public double apply(double p) { return p * 0.1; }
}
```

Technique 3 - Introduce Parameter Object

Group related parameters into a single value object.

Before

```
// Too many parameters - hard to read and easy to mix up
public void createUser(String firstName, String lastName,
                      String email, String phone) { ... }
```

After

```
public class UserRequest {
    public final String firstName, lastName, email, phone;
    public UserRequest(String f, String l, String e, String p) {
        firstName=f; lastName=l; email=e; phone=p;
    }
}
public void createUser(UserRequest req) { ... }
```

Key Rules

- Always have tests in place before refactoring - they are your safety net.
- Extract Method: aim for methods that do exactly one thing with a clear name.
- Replace Conditional with Polymorphism: eliminates fragile switch/if chains on type.
- Introduce Parameter Object: reduces method signatures and groups related data.
- Refactor in small steps and run tests after each step.

Q1. What is the main rule before starting any refactoring?

Ensure you have automated tests confirming current behaviour before making changes.

Q2. When should you use Extract Method?

When a block of code has a clear purpose that can be expressed by a descriptive method name.

Q3. What problem does Replace Conditional with Polymorphism solve?

Removes repeated if/switch statements checking an object type, making code open for extension.

Q4. What is a Parameter Object?

A simple class grouping related method parameters, reducing long parameter lists.

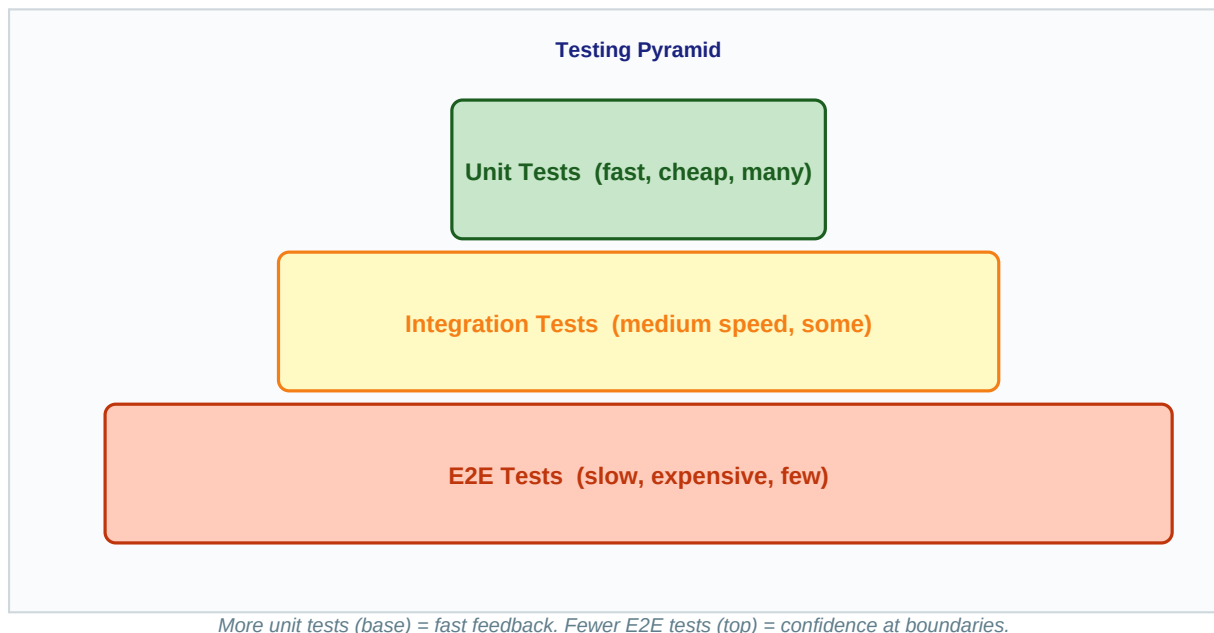
Q5. Does refactoring change the external behaviour of code?

No. By definition, refactoring preserves external behaviour and only improves internal structure.

6 Testing Strategy

Unit / Integration / E2E Pyramid

The Testing Pyramid guides how to allocate test effort. Write many fast unit tests at the base, fewer integration tests in the middle, and a small number of end-to-end tests at the top. This keeps the suite fast and cheap to maintain.



Simple Java Example - Unit Test with JUnit 5

```
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

class OrderTest {

    @Test
    void validOrderHasPositiveTotal() {
        Order o = new Order("c1", 99.0);
        assertTrue(o.isValid());
    }

    @Test
    void zeroTotalIsInvalid() {
        Order o = new Order("c1", 0.0);
        assertFalse(o.isValid());
    }
}
```

Key Rules

- Unit tests cover a single class/method in isolation - use mocks for dependencies.
- Integration tests check that two or more components work together (e.g. service + DB).
- E2E tests exercise the full system from UI/API down to the database.
- Aim for roughly 70% unit, 20% integration, 10% E2E as a starting ratio.
- Fast unit tests give developers immediate feedback during development.

Q1. What is the purpose of the Testing Pyramid?

To guide the right balance of test types: many fast unit tests, fewer slow E2E tests.

Q2. What is the difference between a unit test and an integration test?

Unit tests test one class in isolation; integration tests verify multiple components working together.

Q3. When should mocks be used?

In unit tests to replace external dependencies (DB, email, HTTP) so tests stay fast and isolated.

Q4. Why are E2E tests kept to a minimum?

They are slow, brittle, expensive to maintain, and hard to debug when they fail.

Q5. Which Java testing framework is most common for unit tests?

JUnit 5, often combined with Mockito for mocking dependencies.